shieldify



SMART CONTRACT SECURITY ASSESSMENT

Dark Wallet

SECURITY REVIEW

20 OCTOBER 2026

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

Contents

1 **About Shieldify** 3

2 **Disclaimer** 3

3 **About Dark Wallet** 3

4 **Risk Classification** 4

4.1 Impact 4

4.2 Likelihood 4

5 **Security Review Summary** 4

5.1 Protocol Summary 5

5.2 Scope 5

6 **Findings Summary** 5

7 **Findings** 6

1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3 About Dark Wallet

Dark Wallet is a self-custodial wallet for confidential USDG payments on Robinhood Chain. At its center is `DarkVault`, a single vault contract that holds every deposit and executes every private transfer and withdrawal.

Balances in the vault are encrypted as ElGamal ciphertexts on the Grumpkin curve, so the chain never sees the amount of a private transfer or the balance of an account that has used one. Deposits and withdrawals carry public amounts, and the sender and recipient addresses of every transfer remain public. Dark hides amounts, not the transaction trail. A private transfer or withdrawal is accepted only with a zero-knowledge proof, verified on chain, that the sender owns the balance, that it covers the amount, and, for a transfer, that the amount is correctly encrypted to the recipient's registered key.

At a high level, the main functionalities are:

- Users register a Grumpkin public key in the key registry so that others can pay them
- `deposit()` moves public USDG into the caller's encrypted available balance
- `transfer()` moves a hidden amount into the recipient's encrypted pending balance, backed by a ZK proof
- `applyPending()` folds received transfers into the recipient's spendable balance

- `withdraw()` proves the balance covers the amount and pays out USDG to any address
- An owner behind a 48-hour timelock manages limits within hard-coded ceilings, while a guardian can only tighten limits or pause deposits and transfers; withdrawals and `applyPending()` are never blocked

4 Risk Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	CRITICAL	HIGH	MEDIUM
Likelihood: Medium	HIGH	MEDIUM	LOW
Likelihood: Low	MEDIUM	LOW	LOW

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable, and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5 Security Review Summary

The security review lasted 3 days, with a total of 48 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying nine Informational severity issues. They're mainly related to administrative cap-update coordination, privacy consequences of narrowed transfer limits and sender-supplied hints, proof-invalidation and role-rotation footguns, missing events, strict token-delta handling, and deployment-verification coverage.

The Dark Wallet team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

PROJECT NAME	DARK WALLET
Repository	dark-core
Type of Project	Confidential Payments Vault (ZK-Encrypted Balances)
Security Review Timeline	3 days
Review Commit Hash	379bec5ef8256592b8248895243be97f93fabfd3

5.2 Scope

The following smart contracts were in the scope of the security review:

FILE	NSLOC
src/DarkVaultCore.sol	218
src/DarkVault.sol	80
Total	298

6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Info** issues: 9

ID	TITLE	SEVERITY	STATUS
[I-01]	Previously Scheduled Cap Updates Can Overwrite Later Emergency Tightening	INFO	ACKNOWLEDGED
[I-02]	Singleton Transfer Limits Make Successful Transfer Amounts Publicly Inferable	INFO	ACKNOWLEDGED
[I-03]	Transfer Hint Confidentiality Depends on Sender-Side Encryption	INFO	ACKNOWLEDGED
[I-04]	`applyPending()` on Empty Pending Still Bumps `nonce` - One Call Self-Invalidates All In-Flight Proofs	INFO	ACKNOWLEDGED
[I-05]	`setGuardian()` Is Single-Step - A Mistyped Address Silently Disables the Guardian Role	INFO	ACKNOWLEDGED

ID	TITLE	SEVERITY	STATUS
[I-06]	`recoverERC20()` Emits No Event	INFO	ACKNOWLEDGED
[I-07]	Strict `received == amount` on `deposit()`	INFO	ACKNOWLEDGED
[I-08]	`DarkGrumpkin.inv` : Silent Failure Without Precompile 0x05	INFO	ACKNOWLEDGED
[I-09]	`verify-mainnet.py` Doesn't Cover Verifier/Sibling Bytecode	INFO	ACKNOWLEDGED

7 Findings

I-01

INFO ACKNOWLEDGED

Previously Scheduled Cap Updates Can Overwrite Later Emergency Tightening

Description

`setCaps()` replaces the entire cap configuration without checking whether the configuration has changed since the owner prepared or scheduled the update.

Consequently, an owner operation scheduled before an incident can execute after the guardian tightens the caps and restore the previously proposed, less restrictive values.

Similarly, a guardian transaction prepared from an earlier configuration can lower unrelated fields after an owner raise, because it submits the entire struct.

These transitions remain authorized. The concern is stale administrative intent and emergency-operation coordination.

Location of Affected Code

File: src/DarkVault.sol#L32-L47

```
function setCaps(Caps calldata newCaps) external onlyOwner {
  _checkCeilings(newCaps);
  emit CapsUpdated(_caps, newCaps, msg.sender);
  _caps = newCaps;
}

function tightenCaps(Caps calldata newCaps) external {
  if (msg.sender != _guardian && msg.sender != owner()) revert NotGuardian();
```

```
    Caps memory c = _caps;
    bool ok = newCaps.maxDeposit <= c.maxDeposit
      && newCaps.maxAccountInflow <= c.maxAccountInflow
      && newCaps.maxTransfer <= c.maxTransfer
      && newCaps.tvlCap <= c.tvlCap
      && newCaps.minDeposit >= c.minDeposit
      && newCaps.minTransfer >= c.minTransfer;
    if (!ok) revert NotTightening();
    emit CapsUpdated(c, newCaps, msg.sender);
    _caps = newCaps;
  }
```

Impact

A previously scheduled cap update can undo an emergency restriction implemented through caps alone. It can also increase the permitted exposure during an incident.

An existing pause survives a cap update. Withdrawals and `applyPending()` remain available, and the scheduled operation still requires authorized execution after its timelock.

Proof of Concept

Save as `test/StaleCapsWritePoC.t.sol`.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {DarkBase} from "./DarkBase.t.sol";
import {DarkVault} from "../src/DarkVault.sol";
import {IDarkVault} from "dark-contracts/interfaces/IDarkVault.sol";
import {TimelockController} from
  "@openzeppelin/contracts/governance/TimelockController.sol";

contract StaleCapsWritePoC is DarkBase {
  function test_scheduledSetCapsOverwritesLaterTightening() public {
    address[] memory roles = new address[](1);
    roles[0] = owner;

    TimelockController timelock =
      new TimelockController(48 hours, roles, roles, address(0));

    IDarkVault.Caps memory initial = defaultCaps();
    initial.maxDeposit = 1_000e6;
    initial.maxAccountInflow = 2_500e6;
    initial.maxTransfer = 1_000e6;
    initial.tvlCap = 50_000e6;

    DarkVault target = new DarkVault(
```

```
            address(usdg),
            registry,
            transferV,
            withdrawV,
            address(timelock),
            guard,
            initial
        );

        bytes memory payload =
            abi.encodeCall(DarkVault.setCaps, (defaultCaps()));
        bytes32 salt = keccak256("stale-cap-write");
        uint256 start = block.timestamp;

        vm.prank(owner);
        timelock.schedule(
            address(target), 0, payload, bytes32(0), salt, 48 hours
        );

        vm.warp(start + 24 hours);

        IDarkVault.Caps memory emergency = target.caps();
        emergency.maxDeposit = 0;
        emergency.maxTransfer = 0;

        vm.prank(guard);
        target.tightenCaps(emergency);

        assertEq(target.caps().maxDeposit, 0);
        assertEq(target.caps().maxTransfer, 0);

        vm.warp(start + 48 hours);

        vm.prank(owner);
        timelock.execute(
            address(target), 0, payload, bytes32(0), salt
        );

        assertEq(target.caps().maxDeposit, 2_500e6);
        assertEq(target.caps().maxTransfer, 2_500e6);
        assertEq(target.caps().tvlCap, 250_000e6);
        assertFalse(target.paused());
    }
}
```

Run:

```
forge test --match-contract StaleCapsWritePoC -vv
```

Result: **1 passed**.

Recommendation

For the deployed version:

- Cancel obsolete pending cap updates when emergency restrictions are applied.
- Review current configuration before executing scheduled changes.
- Apply `pause()` alongside cap tightening when deposits and transfers must remain disabled.

For a future version, require an expected configuration version when updating caps and increment that version whenever caps change. Guardian updates should support explicitly selected fields, avoiding accidental changes to unrelated values.

Team Response

Acknowledged.

I-02

INFO ACKNOWLEDGED

Singleton Transfer Limits Make Successful Transfer Amounts Publicly Inferable

Description

The cap setters permit an enabled transfer window with identical minimum and maximum values:

```
minTransfer = maxTransfer = X
```

The guardian can create this configuration through `tightenCaps()` when `X` lies within the existing transfer window. The owner can also configure it through `setCaps()`.

The vault supplies these public limits to the transfer verifier. Because the verifier enforces the inclusive condition `minTransfer <= amount <= maxTransfer`, every successful transfer under this configuration necessarily transfers exactly `X`. The transfer circuit supplies supporting context for this deduction.

This is a privacy consequence of the permitted administrative configuration.

Location of Affected Code

File: src/DarkVault.sol#L38-L47

```
bool ok = newCaps.maxDeposit <= c.maxDeposit
  && newCaps.maxAccountInflow <= c.maxAccountInflow
```

```
    && newCaps.maxTransfer <= c.maxTransfer
    && newCaps.tvlCap <= c.tvlCap
    && newCaps.minDeposit >= c.minDeposit
    && newCaps.minTransfer >= c.minTransfer;

if (!ok) revert NotTightening();

emit CapsUpdated(c, newCaps, msg.sender);
_caps = newCaps;
```

File: src/DarkVaultCore.sol#L250-L251

```
pi[19] = bytes32(uint256(_caps.minTransfer));
pi[20] = bytes32(uint256(_caps.maxTransfer));
```

Impact

Observers can infer the exact amount of every successful transfer proven while the singleton window remains active, despite the transfer ciphertext retaining randomness.

The effect requires an administrative cap change and applies to transfers validated under that configuration. It does not disclose historical hidden amounts or decrypt previously private balances.

Proof of Concept

Save as `test/SingletonTransferWindowPoC.t.sol`.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {DarkBase} from "./DarkBase.t.sol";
import {IDarkVault} from "dark-contracts/interfaces/IDarkVault.sol";

contract SingletonTransferWindowPoC is DarkBase {
  function test_guardianCanEnableSingletonWindow() public {
    IDarkVault.Caps memory c = vault.caps();
    c.minTransfer = 100e6;
    c.maxTransfer = 100e6;

    vm.prank(guard);
    vault.tightenCaps(c);

    assertEq(vault.caps().minTransfer, 100e6);
    assertEq(vault.caps().maxTransfer, 100e6);
    assertFalse(vault.paused());
  }
}
```

Run:

```
forge test --match-contract SingletonTransferWindowPoC -vv
```

Result: **1 passed**.

The test demonstrates that the singleton configuration is accepted. Exact amount inference follows directly from the public bounds and the verifier's inclusive inequalities.

Recommendation

If enabled singleton windows are undesirable, reject `minTransfer == maxTransfer` consistently in the constructor, `setCaps()`, and `tightenCaps()`, while preserving intentionally disabled configurations.

Also disclose the privacy effect of narrowed bounds and warn users before proving transfers under a singleton window.

A minimum spread or ratio can be an additional policy control, but it does not independently guarantee amount confidentiality.

Team Response

Acknowledged.

I-03

INFO ACKNOWLEDGED

Transfer Hint Confidentiality Depends on Sender-Side Encryption

Description

`DarkVaultCore.transfer()` accepts a fixed-length recipient hint and publishes it in `ConfidentialTransfer`. The hint is excluded from the transfer proof's public inputs (built in `_transferInputs()`), so successful proof verification does not attest the hint's encryption, plaintext correctness, or confidentiality.

Under the documented hint format, the sender chooses an ephemeral scalar `k` and derives the shared point:

```
K = k · P_r
```

A sender choosing `k = 1` obtains `K = P_r`, which is publicly available from the registry. Observers can therefore derive the hint encryption key and read its amount and note.

Location of Affected Code

File: src/DarkVaultCore.sol#L156

```
if (hint.length != HINT_LEN) revert BadBlobLength(hint.length, HINT_LEN);
```

The caller-supplied hint is published directly:

File: src/DarkVaultCore.sol#L178-L188

```
emit ConfidentialTransfer(
  msg.sender,
  to,
  s.nonce,
  [ct.c.x, ct.c.y, ct.dSender.x, ct.dSender.y, ct.dRecipient.x, ct.dRecipient.y],
  _ct(s.available),
  _ct(r.pending),
  r.pendingCount,
  hint,
  senderHint
);
```

Impact

A sender can make its own transfer amount and authored note publicly readable through a deliberately weak hint.

The sender already knows these values and can disclose them independently. This observation does not reveal an unknown recipient secret, expose unrelated balances, permit theft, or bypass proof verification.

Integrators should therefore distinguish acceptance of a valid transfer from assurance that its accompanying metadata remains confidential.

Recommendation

Clarify the scoped `transfer()` NatSpec:

- Hints are opaque, sender-supplied metadata excluded from proof verification.
- Contract acceptance does not attest their plaintext correctness or confidentiality.
- Intended clients should generate independent, unpredictable hint randomness.

Team Response

Acknowledged.

I-04

INFO ACKNOWLEDGED

`applyPending()` on Empty Pending Still Bumps `nonce` - One Call Self-Invalidates All In-Flight Proofs

Description

`applyPending()` is a state-mutating call even when `pendingCount == 0`: it still increments `nonce`, overwrites `aeBalance` verbatim, and emits the event. Because `nonce` is bound into transfer and withdrawal public inputs (`pi[4]` / `pi[5]`), a single empty `applyPending()` call, which always succeeds, instantly kills **every** transfer/withdraw proof the caller has already generated but not yet submitted.

The trap: an empty apply looks like a harmless no-op (nothing was folded), yet it permanently changes the account's proof context. A user who generates a withdraw proof and then calls `applyPending()` (e.g., to refresh `aeBalance`, or via a wallet button that always applies) bricks their own proof.

Location of Affected Code

File: src/DarkVaultCore.sol#L121-L142

```
function applyPending(uint64 expectedPendingCount, bytes calldata aeBalance) external nonReentrant {
  Account storage a = _accounts[msg.sender];
  _requireRegistered(msg.sender);
  // ...
  uint64 applied = a.pendingCount;
  if (applied < expectedPendingCount) revert PendingChanged(expectedPendingCount, applied);
  _checkAe(aeBalance);

  (a.available.c.x, a.available.c.y) =
    DarkGrumpkin.add(a.available.c.x, a.available.c.y, a.pending.c.x, a.pending.c.y);
  (a.available.d.x, a.available.d.y) =
    DarkGrumpkin.add(a.available.d.x, a.available.d.y, a.pending.d.x, a.pending.d.y);
  delete a.pending;
  a.pendingCount = 0;
  a.nonce += 1;
  a.aeBalance = aeBalance;

  emit PendingApplied(msg.sender, a.nonce, applied, _ct(a.available));
}
```

The nonce increment and the `aeBalance` write are unconditional, not gated on `pendingCount > 0`.

Impact

Self-harm only: the caller controls their own timing, and a correct client simply re-proves. But the failure mode is silent: the empty call succeeds and looks inert, while in-flight proofs die with `InvalidProof` and the user's gas burns on-chain. In a wallet that auto-applies pending (or retries on stale UI state), this produces a confusing "my valid proof keeps reverting" loop.

Proof of Concept

File: `test/EmptyApplySelfInvalidationPoC.t.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {DarkBase} from "./DarkBase.t.sol";
import {IDarkVault} from "dark-contracts/interfaces/IDarkVault.sol";

/// @notice applyPending on EMPTY pending still bumps nonce -> self-invalidates
///         any in-flight proof bound to the old nonce.
contract EmptyApplySelfInvalidationPoC is DarkBase {
  function test_emptyApplyBumpsNonce() public {
    deposit(alice, 1e6);
    uint64 n0 = vault.getAccount(alice).nonce;

    bytes memory ae = new bytes(56);
    vm.prank(alice);
    vault.applyPending(0, ae);      // pendingCount == 0, no revert

    IDarkVault.AccountView memory v = vault.getAccount(alice);
    assertEq(v.nonce, n0 + 1);      // nonce moved despite nothing applied
    assertEq(v.pendingCount, 0);
    assertEq(v.aeBalance, ae);      // aeBalance overwritten verbatim
    // Any transfer/withdraw proof generated against nonce n0 is now dead.
  }
}
```

Recommendation

Consider one of the following:

- Early-return when `pendingCount == 0` (no nonce bump, no `aeBalance` write, no event). This removes the footgun entirely and saves gas.
- Document that `applyPending()` is always state-mutating and must never be called speculatively; the SDK should gate the call on `pendingCount > 0`.

Team Response

Acknowledged.

I-05

INFO ACKNOWLEDGED

`setGuardian()` Is Single-Step - A Mistyped Address Silently Disables the Guardian Role

Description

`setGuardian()` writes the role in a single call. There is no propose/accept step, unlike `Ownable2Step`, which the contract already uses for ownership transfer. A mistyped or unreachable guardian address is accepted instantly and emits a normal `GuardianUpdated` event: from that moment `pause()` and `tightenCaps()` are dead for everyone (the old guardian is overwritten; the new one can never call). Recovery requires the owner to rotate again, behind the 48h timelock.

The constructor similarly performs no sanity check: `guardian_ == owner_` deploys successfully, collapsing the intended two-role separation (emergency brake vs. administrative control held by one key).

Location of Affected Code

Single-step write, only `!= address(0)` checked:

File: src/DarkVault.sol#L49-L53

```
function setGuardian(address guardian_) external onlyOwner {
    if (guardian_ == address(0)) revert ZeroAddress();
    emit GuardianUpdated(_guardian, guardian_);
    _guardian = guardian_;
}
```

No `guardian_ != owner_` check in the constructor:

File: src/DarkVault.sol#L26-L28

```
        address guardian_,
        Caps memory initialCaps
    ) DarkVaultCore(usdg_, registry_, transferVerifier_, withdrawVerifier_, owner_, guardian_, initialCaps) {}
```

`pause()` and `tightenCaps()` revert with `NotGuardian` for the previous guardian:

File: src/DarkVault.sol#L56-L59

```
function pause() external {
  if (msg.sender != _guardian && msg.sender != owner()) revert NotGuardian();
  _pause();
}
```

Impact

Operational risk, not an exploit: the trusted owner fumbles one address and the emergency brake is bricked for up to 48h, exactly the window in which the guardian exists to act. `guardian == owner` at construction silently defeats the documented two-role model.

Proof of Concept

File: `test/GuardianRotationPoC.t.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {DarkBase} from "./DarkBase.t.sol";
import {IDarkVault} from "dark-contracts/interfaces/IDarkVault.sol";
import {DarkVault} from "../src/DarkVault.sol";

/// @notice Single-step guardian rotation: mistyped address silently kills the
///         role; ctor also accepts guardian == owner (role separation collapses).
contract GuardianRotationPoC is DarkBase {
  function test_mistypedGuardianSilentlyDead() public {
    vm.prank(owner);
    vault.setGuardian(address(0xDEAD));  // typo'd address — accepted instantly
    assertEq(vault.guardian(), address(0xDEAD));

    vm.prank(guard);
    vm.expectRevert(IDarkVault.NotGuardian.selector);
    vault.pause();                // old guardian dead; 0xDEAD can never call
  }

  function test_ctorAllowsGuardianEqualsOwner() public {
    DarkVault v = new DarkVault(
      address(usdg), registry, transferV, withdrawV, owner, owner, defaultCaps()
    );
    assertEq(v.guardian(), owner);      // deployed — role separation collapsed
  }
}
```

Recommendation

- Use a two-step rotation (`setGuardian()` → `acceptGuardian()`), mirroring the `Ownable2Step` pattern already in use for ownership. With an accept step in place, additional address guardrails (such as requiring the new guardian to have code or to match a known Safe) are not needed.
- Add `require(guardian_ != owner_)` in the constructor to preserve role separation.

Team Response

Acknowledged.

I-06

INFO ACKNOWLEDGED

`recoverERC20()` Emits No Event

Description

`recoverERC20()` , the only privileged value-moving write, emits no event. Indexers and off-chain monitors cannot see sweeps.

Location of Affected Code

File: src/DarkVault.sol#L65-L69

```
function recoverERC20(address token, address to, uint256 amount) external onlyOwner {
  if (token == _usdg) revert CannotRecoverUSDG();
  if (to == address(0)) revert ZeroAddress();
  IERC20(token).safeTransfer(to, amount);
}
```

Recommendation

Emit a dedicated `ERC20Recovered(token, to, amount)` event.

Team Response

Acknowledged.

I-07

INFO ACKNOWLEDGED

Strict `received == amount` **on** `deposit()`

Description

The delta guard reverts when a token credits **more** than `amount` inside `transferFrom()` (a surplus-crediting USDG upgrade would make all deposits revert, even though the vault is better off). A `received < amount` check keeps the intended fee-on-transfer protection while tolerating surplus, and `amount`-credited accounting stays solvent since the surplus simply joins the stranded pool.

Location of Affected Code

File: src/DarkVaultCore.sol#L105-L108

```
uint256 before = token.balanceOf(address(this));
token.safeTransferFrom(msg.sender, address(this), amount);
uint256 received = token.balanceOf(address(this)) - before;
if (received != amount) revert UnexpectedTransferAmount(amount, received);
```

Proof of Concept

File: `test/DepositStrictDeltaPoC.t.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {DarkBase} from "./DarkBase.t.sol";
import {IDarkVault} from "dark-contracts/interfaces/IDarkVault.sol";
import {DarkVault} from "../src/DarkVault.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

/// @dev Token delivering a 1% bonus to the recipient inside transferFrom.
contract BonusUSDG is ERC20 {
  constructor() ERC20("Bonus USDG", "BUSDG") {}
  function decimals() public pure override returns (uint8) { return 6; }
  function mint(address to, uint256 amount) external { _mint(to, amount); }
  function transferFrom(address from, address to, uint256 amount)
    public override returns (bool)
  {
    bool ok = super.transferFrom(from, to, amount);
    _mint(to, amount / 100);
    return ok;
  }
}
```

```
contract DepositStrictDeltaPoC is DarkBase {
  function test_surplusCreditRevertsDeposit() public {
    BonusUSDG bonus = new BonusUSDG();
    DarkVault v = new DarkVault(
      address(bonus), registry, transferV, withdrawV, owner, guard, defaultCaps()
    );
    bonus.mint(alice, 1e6);
    vm.prank(alice);
    bonus.approve(address(v), 1e6);

    vm.expectRevert(
      abi.encodeWithSelector(IDarkVault.UnexpectedTransferAmount.selector, 1e6, 1.01e6)
    );
    vm.prank(alice);
    v.deposit(1e6, "");     // vault got MORE — still reverts
  }
}
```

Recommendation

Change the deposit delta check from `received == amount` to `received >= amount` to allow surplus-crediting tokens while continuing to reject fee-on-transfer tokens. Continue accounting for deposits using the requested amount, with any surplus remaining as unaccounted vault balance.

Team Response

Acknowledged.

I-08

INFO ACKNOWLEDGED

`DarkGrumpkin.inv` : Silent Failure Without Precompile 0x05

Description

On a chain lacking EIP-198, the `staticcall` to `0x05` returns success with empty returndata, so `out` reads stale memory and silently corrupted points propagate through all four arithmetic paths into stored state. This is latent on chain 4663 (the precompile is present), but fires on any future deployment chain without it.

Additionally, the `DarkGrumpkin.inv()` doc comment claims a revert on `a == 0` that does not exist (unreachable today).

Location of Affected Code

File: lib/dark-contracts/libraries/DarkGrumpkin.sol#L130-L143

```
/// @dev a^(P-2) mod P via the modexp precompile (0x05). Reverts on a == 0 (never called with 0).
function inv(uint256 a) private view returns (uint256 out) {
  assembly ("memory-safe") {
    let m := mload(0x40)
    mstore(m, 0x20)
    mstore(add(m, 0x20), 0x20)
    mstore(add(m, 0x40), 0x20)
    mstore(add(m, 0x60), a)
    mstore(add(m, 0x80), sub(P, 2))
    mstore(add(m, 0xa0), P)
    if iszero(staticcall(gas(), 0x05, m, 0xc0, m, 0x20)) { revert(0, 0) }
    out := mload(m)
  }
}
```

Recommendation

Assert a non-empty returndata after the `staticcall`, and add a check for the presence of precompile `0x05` to the deployment checklist.

Team Response

Acknowledged.

I-09 | INFO ACKNOWLEDGED

`verify-mainnet.py` Doesn't Cover Verifier/Sibling Bytecode

Description

The script proves the vault's own bytecode and role addresses, but the verifiers are split contracts with **hardcoded** `DELEGATECALL` into shared sibling code (`0x97db...c085`, `0x4380...8e4c`). A swapped verifier or sibling would pass the current check.

Location of Affected Code

File: script/verify-mainnet.py

Recommendation

Extend the verification to the verifier and sibling bytecode for a complete deployment attestation.

Team Response

Acknowledged.



Your security partner, for the long term

SHIELDIFY.ORG · X.COM/SHIELDIFYSEC · LINKEDIN